

Práctica 6: Sistema Multiagente Unificado de Gestión Financiera Personal



Autores:

Diego Esclarín (51729611N)
Sofía Contreras (09848471D)

Aplicaciones de Inteligencia Artificial
Grado en Ingeniería en Inteligencia Artificial
Universidad Rey Juan Carlos
Curso 2025–26

Índice

1. Introducción	5
2. Contexto	7
2.1. Punto de partida: prácticas previas	7
2.2. Marco teórico	7
2.3. Entorno de ejecución	8
3. Estado del Arte	9
3.1. Asistentes financieros personales	9
3.2. Arquitecturas multiagente con LLM	9
3.3. Reconocimiento facial y <i>liveness</i>	9
3.4. OCR de recibos y tickets	10
3.5. Detección de anomalías financieras	10
4. Propuesta	11
4.1. Topología del sistema y reglas	11
4.2. Doble modo del orquestador	12
4.3. Sub-agente Analyst	13
4.4. Sub-agente Registrar y revisión de pendientes	13
4.5. Sub-agente Security	14
4.6. Persistencia y migraciones ligeras	14
4.7. API REST y frontal Next.js	15
5. Conclusiones	17

Índice de figuras

1.	Topología del sistema multiagente de P6.	12
2.	Flujo de un turno en el orquestador: doble modo <code>_route</code> / <code>_narrate</code> y bucle controlado.	13
3.	Pantalla de autenticación con captura de webcam para verificación biométrica.	15
4.	Pantalla de chat: el orquestador narra en español el resultado devuelto por el sub-agente correspondiente.	16
5.	Vista de transacciones pendientes con acciones de aprobar y rechazar. . . .	16

1. Introducción

La gestión financiera es perfecta para aplicar la IA debido a la diversidad de sus datos: el historial de movimientos bancarios, las fotos de los tickets físicos y las preguntas que el usuario hace sobre sus propias finanzas. En las prácticas anteriores de la asignatura se abordó cada uno de estos retos por separado: predicción de gasto futuro con modelos clásicos de *machine learning* (P1), clasificación automática de transacciones por categoría (P2), extracción de importes mediante OCR sobre tickets (P3), exploración del paradigma de agentes con LangGraph (P4) y autenticación biométrica facial junto a detección de anomalías (P5).

El objetivo de la **Práctica 6** es **unificar todas las funcionalidades** en una única aplicación multiagente, persistente y multiusuario, que sustituye el conjunto separado de *scripts* y prototipos por un sistema cohesionado, accesible vía API REST y frontal web, gobernado por un orquestador basado en LLM. La aplicación construye una experiencia de *copiloto financiero personal* donde un único punto de entrada conversacional permite al usuario registrar gastos manualmente o por foto, consultar analíticas, recibir alertas ante movimientos anómalos, autenticarse mediante reconocimiento facial y revisar transacciones marcadas para revisión.

Más concretamente, los objetivos son: (i) integrar las capacidades de P1–P5 bajo una arquitectura común coordinada por un grafo de agentes; (ii) implementar persistencia real en base de datos relacional con soporte multiusuario y aislamiento estricto de datos; (iii) exponer el sistema mediante una API REST autenticada con JWT y un frontal Next.js; (iv) garantizar el cumplimiento del RGPD en cuanto a consentimiento biométrico, redacción de notificaciones y cifrado de datos sensibles; y (v) demostrar que la arquitectura propuesta es robusta.

2. Contexto

2.1. Punto de partida: prácticas previas

La Práctica 6 no es un sistema nuevo creado desde cero, sino que integra y mejora los módulos desarrollados en las prácticas anteriores. La tabla 1 muestra qué elementos se han aprovechado de cada una.

Origen	Código reutilizado	Ubicación en P6
P1	Predictores RF, HGB y ARIMA	<code>analyst/forecasters.py</code>
P2	<code>FinancialClassifier</code> (TF-IDF + SGD)	<code>registrar/classifier.py</code>
P3	Motor OCR (PaddleOCR + GB scoring)	<code>registrar/ocr_engine.py</code>
P4	Esqueleto LangGraph y analíticas	<code>orchestrator/graph.py</code> , <code>analyst/analytics.py</code>
P5	<code>EncryptedEmbeddingStore</code> y <code>AccessController</code>	<code>utils/security.py</code>
P5	Pipeline biométrico (MTCNN + FaceNet + DenseNet201)	<code>security/biometrics.py</code>
P5	<code>FinancialAnomalyDetector</code> (IsolationForest + 3σ)	<code>security/anomaly_detector.py</code>

Cuadro 1: Reutilización de prácticas anteriores en P6.

Asimismo, se han descartado varios activos: la interfaz Streamlit de P4 (sustituida por un frontal Next.js + API FastAPI), las CLI `argparse` de P1–P3 y P5 (reemplazadas por endpoints HTTP), `deepface`/ArcFace (eliminada para evitar arrastrar TensorFlow), `pmdarima` (sustituida por `statsmodels.tsa`, con menos problemas de compilación) y EasyOCR como motor secundario de P3 (PaddleOCR pasa a ser el único motor).

2.2. Marco teórico

La práctica integra cuatro técnicas diferentes:

- **Sistemas multiagente coordinados por LLM.** El paradigma adoptado es el de **grafo de agentes** de LangGraph [1], donde cada nodo recoge una capacidad especializada (Security, Registrar, Analyst) y un orquestador con LLM toma decisiones de *routing* mediante *structured output*. La separación entre razonamiento (LLM en el orquestador) y ejecución (modelos clásicos deterministas en los sub-agentes) es deliberada y constituye una de las decisiones de diseño centrales.
- **Aprendizaje automático clásico.** Los sub-agentes especializados utilizan modelos no-LLM: clasificación lineal con `SGDClassifier` sobre n-gramas de carácter para la categorización de descripciones; *Random Forest*, *Histogram Gradient Boosting* y ARIMA para previsión de gasto; *IsolationForest* y reglas 3σ para detección de anomalías financieras. Esta elección reduce drásticamente la huella de memoria respecto a alternativas basadas en *transformers* y aporta determinismo y latencias predecibles.

- **Visión por computador.** El registro por imagen utiliza **PaddleOCR** [2] para la detección y reconocimiento de texto, seguido de un clasificador de *Gradient Boosting* que puntúa cada *token* numérico para identificar el importe total del recibo. La autenticación biométrica encadena **MTCNN** [3] (detección y alineamiento de rostros), **FaceNet** [4] (InceptionResnetV1 preentrenado en VGGFace2 para generar *embeddings* de 512 dimensiones) y **DenseNet201** para verificación de *liveness* (anti-spoofing).
- **Seguridad y privacidad por diseño.** Cifrado simétrico Fernet (AES-128 CBC + HMAC-SHA256) con clave derivada por PBKDF2-HMAC-SHA256 (310 000 iteraciones) sobre los *embeddings* biométricos; **bcrypt** para *passphrases*; tokens JWT firmados para sesiones; *rate limiting* con bloqueo tras 5 intentos en 5 minutos; control de envío de notificaciones según preferencia RGPD del usuario.

2.3. Entorno de ejecución

El sistema está pensado para ejecutarse en una máquina con Python 3.12, sin necesidad de GPU. La pieza más exigente es **PaddleOCR** (~500 MB de pesos al primer uso) y el *stack* biométrico (~150 MB). Ambos se cargan de forma **perezosa** en la primera operación que los requiera, de manera que el arranque del orquestador es prácticamente instantáneo. La base de datos por defecto es **SQLite local** (data/p6.db) para minimizar la fricción en desarrollo, con soporte para **PostgreSQL 15** mediante **docker-compose**. No obstante, para el despliegue final se ha optado por una instancia gestionada en **Supabase**, garantizando así la persistencia y disponibilidad del sistema en la nube.

3. Estado del Arte

3.1. Asistentes financieros personales

El mercado de asistentes financieros ha evolucionado en tres oleadas. La primera, encarnada en aplicaciones como *Mint* o *YNAB*, se basaba en **reglas explícitas** y categorización manual: el usuario etiquetaba sus transacciones y la herramienta agregaba dashboards. La segunda oleada introdujo **aprendizaje supervisado clásico** para sugerir categorías y detectar patrones (clasificadores lineales sobre descripciones, segmentación temporal de series). La tercera y actual oleada integra **LLM conversacionales** (Cleo, Copilot Money, Albert) capaces de responder preguntas en lenguaje natural sobre el estado financiero del usuario.

Esta práctica se sitúa en esta tercera oleada, pero con dos diferencias relevantes respecto al estado del arte comercial: (i) el LLM **no se encarga del cálculo** sino únicamente del enrutamiento y la narración, delegando los *analytics* en código determinista (decisión motivada por evitar alucinaciones numéricas); y (ii) el sistema es **auto-alojable y de código abierto**, sin enviar transacciones a servicios cerrados.

3.2. Arquitecturas multiagente con LLM

Los *frameworks* de orquestación de agentes se han desarrollado en los últimos dos años: **LangGraph** [1], **CrewAI**, **AutoGen** y **LlamaIndex Agents** representan enfoques con compromisos distintos. LangGraph se ha consolidado como referencia para flujos donde el control del estado y la persistencia entre turnos son críticos: el grafo se modela explícitamente como un autómata con *nodes* y *edges*, el estado se serializa con **MemorySaver** o **PostgresSaver**, y se habilitan *structured outputs* compatibles con cualquier proveedor LLM.

La literatura reciente [5] identifica tres patrones de diseño en sistemas multiagente: *router-based* (un agente central decide a quién delegar), *hierarchical* (jerarquías de planificadores y ejecutores) y *collaborative* (varios agentes negocian en paralelo). P6 implementa una variante *router-based* estricta donde **únicamente el orquestador habla con el usuario** y los sub-agentes devuelven *dataclasses* Pydantic tipadas, no texto libre. Esta restricción simplifica el control de errores, evita bucles de delegación y reduce el coste por turno.

3.3. Reconocimiento facial y *liveness*

El estado del arte en *embeddings* faciales gira alrededor de tres grupos de pérdidas: **Triplet Loss** (FaceNet [4], 2015), **ArcFace** [6] (2019) y **CosFace**. ArcFace ofrece resultados ligeramente superiores en benchmarks (LFW, MegaFace), pero su implementación más extendida (**deepface**) arrastra TensorFlow como dependencia. P6 opta por **FaceNet** a través de **facenet-pytorch**, sacrificando un margen pequeño de precisión a cambio de evitar la coexistencia de tres frameworks de *deep learning* (PaddlePaddle + PyTorch + TensorFlow) en una única imagen Docker.

En cuanto a *liveness*, los enfoques modernos se dividen en análisis de textura, que buscan patrones de impresión o pantalla (LBP, redes preentrenadas en CASIA-FASD/Replay-Attack) y **basados en señales fisiológicas** (rPPG, parpadeo, movimiento 3D). P6 emplea un clasificador binario sobre **DenseNet201** preentrenado en ImageNet, sin *fine-tuning* específico anti-spoof; la *roadmap* contempla la mejora de este componente con datasets dedicados como NUAA o Replay-Attack.

3.4. OCR de recibos y tickets

La extracción de información estructurada de recibos es un problema clásico que combina detección de texto, reconocimiento óptico y *post-processing* semántico. Las soluciones predominantes son: **Tesseract** (clásico, sin *deep learning*), **EasyOCR** (PyTorch, multilingüe), **PaddleOCR** [2] (PaddlePaddle, modelos compactos PP-OCRv4) y servicios cloud (Google Document AI, AWS Textract). P6 hereda de la P3 un *pipeline* con PaddleOCR como motor único, complementado con un clasificador de *Gradient Boosting* que aprende a reconocer cuál de los *tokens* numéricos extraídos por el OCR corresponde al importe total. El *dataset* de entrenamiento original (CORD, en won surcoreanos) se extendió con regex específicos para formato europeo (25,50, 91,88€).

3.5. Detección de anomalías financieras

Los sistemas anti-fraude bancarios profesionales utilizan modelos sofisticados (LSTM, Graph Neural Networks sobre redes de transacciones, *ensembles* con cientos de *features*). En el contexto académico de esta práctica, P6 combina **IsolationForest** (no supervisado, sensible a outliers multivariantes) con reglas 3σ por categoría y un control de categoría. Es más sencillo y, sobre todo, **interpretable**: cada veredicto del Security viene acompañado de una lista de razones legibles que se traduce en una notificación informativa para el usuario.

4. Propuesta

La propuesta de P6 es una **aplicación multiagente unificada** compuesta por: un grafo LangGraph con un orquestador y tres sub-agentes especializados; una capa de persistencia portable (SQLite o PostgreSQL y Supabase); una API REST FastAPI con autenticación JWT; un frontal Next.js con captura de webcam; y un conjunto de utilidades transversales (cifrado, notificaciones, *logging* estructurado, configuración Pydantic).

4.1. Topología del sistema y reglas

La figura 1 ilustra el diseño de alto nivel. El sistema obedece a seis **reglas esenciales** que se mantienen invariantes a lo largo de toda la implementación:

1. **Solo el orquestador habla con el usuario.** Los sub-agentes devuelven `dataclasses` Pydantic tipadas (`AnalysisReport`, `SecurityVerdict`, `RegistryResult`); nunca texto libre.
2. **Solo el orquestador usa LLM.** Security, Registrar y Analyst son deterministas: combinan modelos clásicos de *machine learning* con reglas explícitas.
3. **Aislamiento de estado.** Cada sub-agente escribe en su propio *slot* del `OrchestratorState`; los *slots* se resetean entre turnos.
4. **Bucle controlado.** El flujo es *routing* → sub-agente → narración → END, con un cinturón anti-bucles de 6 iteraciones.
5. **Notificaciones disparadas en el agente de origen**, nunca desde el frontal.
6. **Aislamiento de datos por usuario.** Cada validación de transacción se hace contra el histórico del propio usuario, sin *fallback* al CSV de demostración.

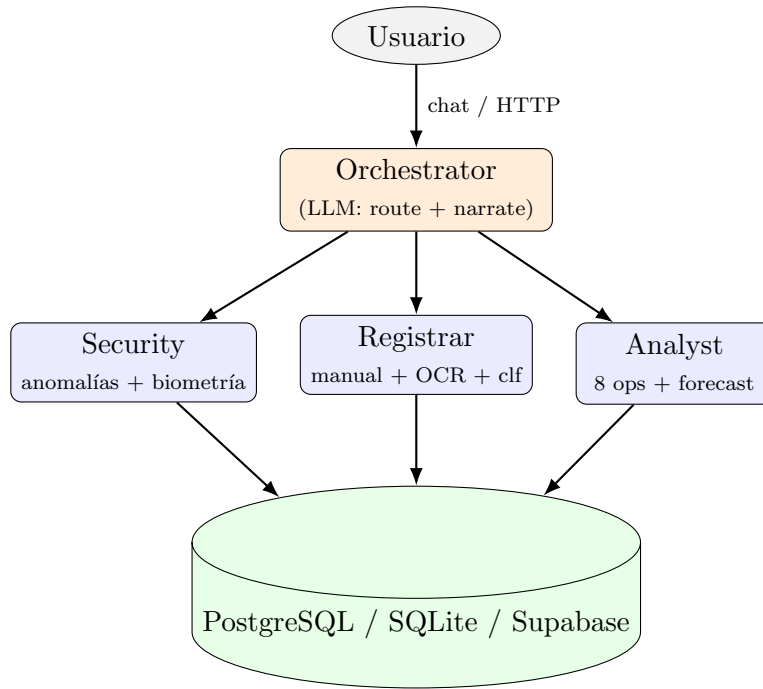


Figura 1: Topología del sistema multiagente de P6.

4.2. Doble modo del orquestador

El orquestador resuelve un problema sutil pero crítico: cuando el LLM utiliza *structured output* para decidir a qué sub-agente delegar, tiende a ignorar los *slots* que ya tienen datos y solicitar nuevas delegaciones, generando bucles. La solución consiste en **separar las dos llamadas al LLM por responsabilidad**:

- `_route` (*structured output*). Se invoca cuando ningún *slot* de sub-agente tiene datos. Devuelve un objeto `OrchestratorDecision` con el campo `next_agent` y los argumentos de la operación. No emite texto al usuario.
- `_narrate` (*invoke plano*). Se invoca cuando algún *slot* ya tiene resultado. Recibe el contenido del *slot* y produce una respuesta en español natural para el usuario.

La detección de modo es trivial: la función `_has_subagent_data(state)` comprueba si `analysis_report`, `security_verdict` o `registry_result` tienen datos. Adicionalmente, los argumentos generados por el LLM se filtran con `_safe_kwargs(func, args)`, que descarta cualquier *keyword argument* alucinado contrastándolo contra `inspect.signature(func)`. La figura 2 resume el flujo completo de un turno.

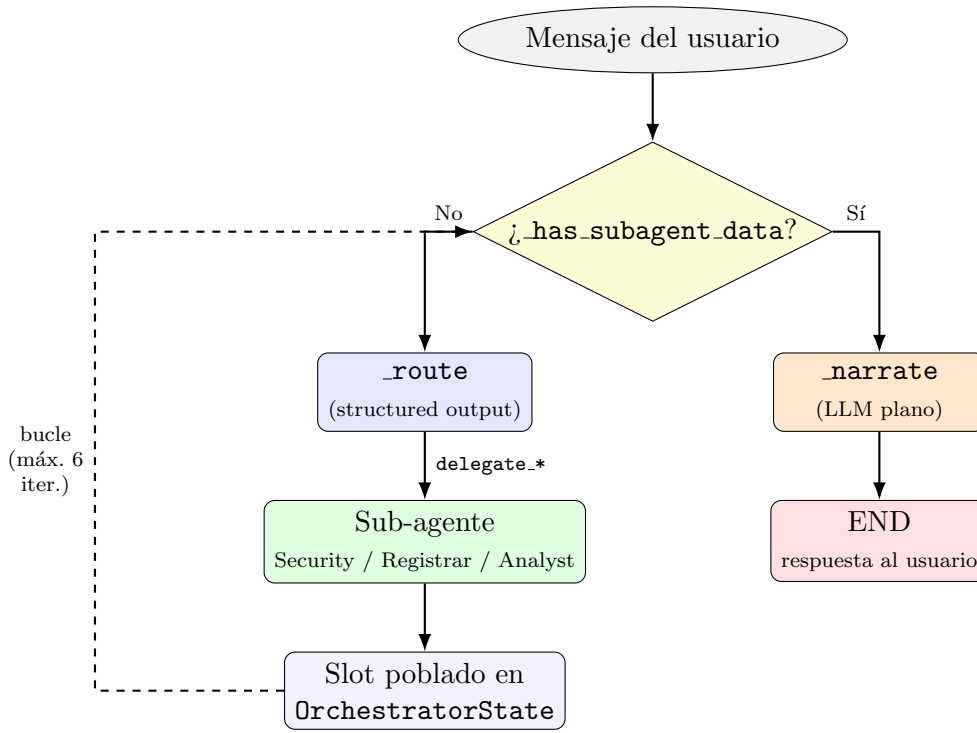


Figura 2: Flujo de un turno en el orquestador: doble modo `_route` / `_narrate` y bucle controlado.

4.3. Sub-agente Analyst

El **Analyst** es el componente con mayor diversidad operativa: expone **nueve operaciones** sobre el histórico del usuario, cargado desde la base de datos (con *fallback* al CSV de demostración):

- `monthly_summary`: gasto, ingreso y balance del mes corriente.
- `category_breakdown`: desglose por área (*vivienda, alimentación, etc.*).
- `spending_trends`: evolución temporal con detección de tendencia.
- `savings_rate`: tasa de ahorro acumulada.
- `detect_anomalies`: aplicación del detector 3σ + IsolationForest.
- `recurring_expenses`: identificación de pagos repetitivos.
- `recent_transactions`: últimas n transacciones aceptadas.
- `predict_next_month`: previsión con *ensemble* (RF + HGB + ARIMA).
- `check_goals`: comparación contra objetivos definidos por el usuario.

El *forecaster* entrena los tres modelos (*Random Forest, Histogram Gradient Boosting* y *ARIMA*) sobre la serie mensual agregada y combina sus predicciones por promedio simple, una elección suficientemente robusta dada la cantidad de datos disponible (12–24 puntos típicamente).

4.4. Sub-agente Registrar y revisión de pendientes

El **Registrar** expone dos operaciones de alta y tres de revisión:

- `add_manual_transaction`: alta a partir de descripción + importe + fecha. La descripción se clasifica automáticamente con `FinancialClassifier` (TF-IDF + n-gramas de carácter + `SGDClassifier`); a continuación, Security valida la transacción contra el histórico.
- `add_from_image`: alta desde una imagen. El `OCRTotalExtractor` (PaddleOCR + *Gradient Boosting* para identificar el *token* de total) extrae el importe; Security valida; la descripción se infiere si no se aporta.
- `list_pending_reviews`, `confirm_pending`, `reject_pending`: gestión del flujo de revisión (*ciclo 8*). Cuando Security marca una transacción como `challenge`, ésta se persiste con `status='pending'` y queda disponible para que el usuario la confirme o rechace en un turno posterior, en lugar de perderse al cerrar la conversación.

Para que las transacciones pendientes **no contaminen los analytics**, el `load_from_db` del Analyst recibe un parámetro `only_accepted=True` por defecto.

4.5. Sub-agente Security

El Security se compone de dos capas:

Detección de anomalías financieras. Adaptación de `FinancialAnomalyDetector` (P5) al esquema de P6: usa las columnas `Amount_clean`, `Date_parsed` y `Area` (multilabel). Combina `IsolationForest` con reglas 3σ por categoría y una regla específica de *categoría nunca antes vista*. Para evitar falsos positivos en usuarios nuevos, no devuelve `challenge` si el historial tiene menos de 5 transacciones, y la regla de categoría nueva sólo se activa cuando el usuario ya tiene al menos 3 categorías diferentes.

Pipeline biométrico. `BiometricPipeline` encadena `MTCNN` (detección y alineamiento), `FaceNet` (*embedding* de 512D, normalizado L2) y `DenseNet201` (*liveness* con umbral 0,65). Las funciones `register_user` y `login_user` son las únicas operaciones del Security accesibles **sólo** desde los endpoints HTTP (no desde el chat, que no puede subir bytes de imagen). `register_user` exige consentimiento RGPD explícito, hasha la *passphrase* con `bcrypt` (API nativa, tras detectar incompatibilidad entre `passlib 1.7.4` y `bcrypt 4.x`) y persiste el *embedding* cifrado con `Fernet+PBKDF2-HMAC-SHA256`. `login_user` aplica una *cosine similarity* mínima de 0,60 frente al *embedding* de referencia y un `AccessController` que bloquea la cuenta tras 5 intentos fallidos en 5 minutos.

4.6. Persistencia y migraciones ligeras

El esquema relacional comprende cinco tablas: `users`, `user_settings`, `transactions`, `goals` y `events`. Para garantizar portabilidad entre PostgreSQL y SQLite se evitan los tipos específicos de `dialects.postgresql` (`UUID`, `ARRAY`, `JSONB`) y se usan los tipos genéricos `sqlalchemy.Uuid` y `sqlalchemy.JSON`. Por defecto, si `DATABASE_URL` está vacía o contiene un valor inválido (como el placeholder ...), un validador Pydantic conmuta a SQLite local sin intervención del desarrollador.

Una limitación conocida de `Base.metadata.create_all()` es que no añade columnas a tablas preexistentes, lo que produjo *schema drift* cuando se añadieron `status` y `anomaly_reasons` a `transactions`. Como solución intermedia (sin pasar todavía por Alembic) se implementó `apply_lightweight_migrations` en `init_db.py`: inspecciona el esquema real, detecta columnas declaradas que faltan y aplica `ALTER`

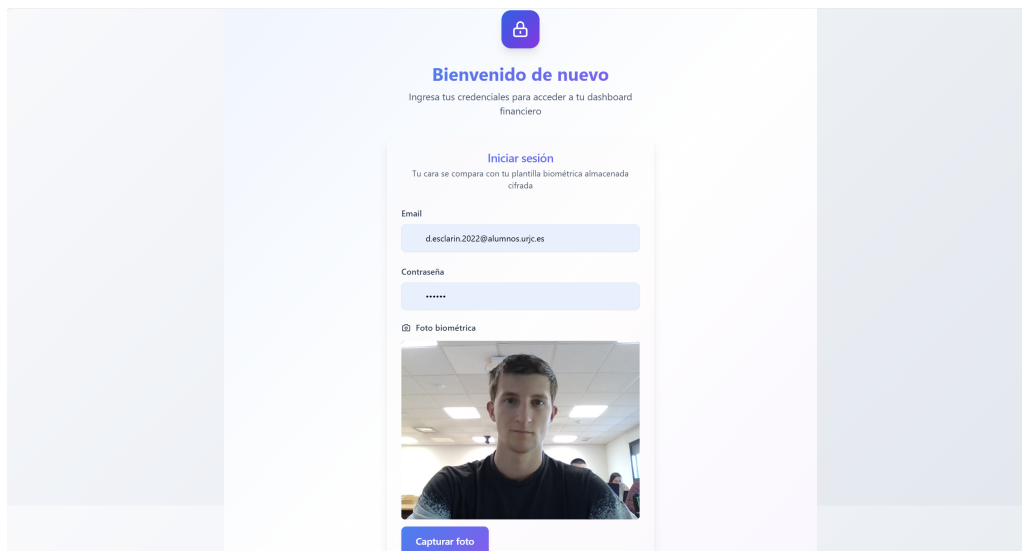


Figura 3: Pantalla de autenticación con captura de webcam para verificación biométrica.

TABLE ADD COLUMN de forma idempotente y no destructiva, preservando los datos existentes.

4.7. API REST y frontal Next.js

La **API FastAPI** expone los siguientes *endpoints*:

- POST `/auth/register` y POST `/auth/login`: multipart/form-data con email, *passphrase* y foto. Devuelven JWT firmado con `settings.jwt_secret`.
- POST `/chat`: invocación del grafo LangGraph; admite `session_id` para conversaciones multi-turno con `MemorySaver`.
- POST `/transactions`: alta manual.
- GET `/transactions/pending`, POST `/transactions/pending/{id}/confirm`, DELETE `/transactions/pending/{id}`: revisión de pendientes.

El grafo se construye una sola vez al arrancar (`_GRAPH = build_graph()`) y se reutiliza entre peticiones, de manera que el `MemorySaver` mantiene estado por usuario y sesión. Los errores se mapean a códigos HTTP semánticos: 400 (`deny` por consentimiento), 401 (*passphrase* o token inválido), 404 (transacción no encontrada), 409 (doble *confirm*).

El **frontal Next.js 14** (App Router, TypeScript, Tailwind) ofrece páginas para *login*, *register*, *chat*, lista de pendientes y configuración. La captura de webcam se hace con `navigator.mediaDevices.getUserMedia` y un `<canvas>` que serializa la imagen a Blob JPEG; se libera el *stream* al desmontar el componente. El JWT se almacena en `localStorage`; las páginas protegidas comprueban su presencia en `useEffect` y redirigen a `/login` si falta.

Las figuras 3, 4 y 5 muestran respectivamente la pantalla de autenticación con captura biométrica, una conversación con el orquestador y la vista de transacciones pendientes de revisión.

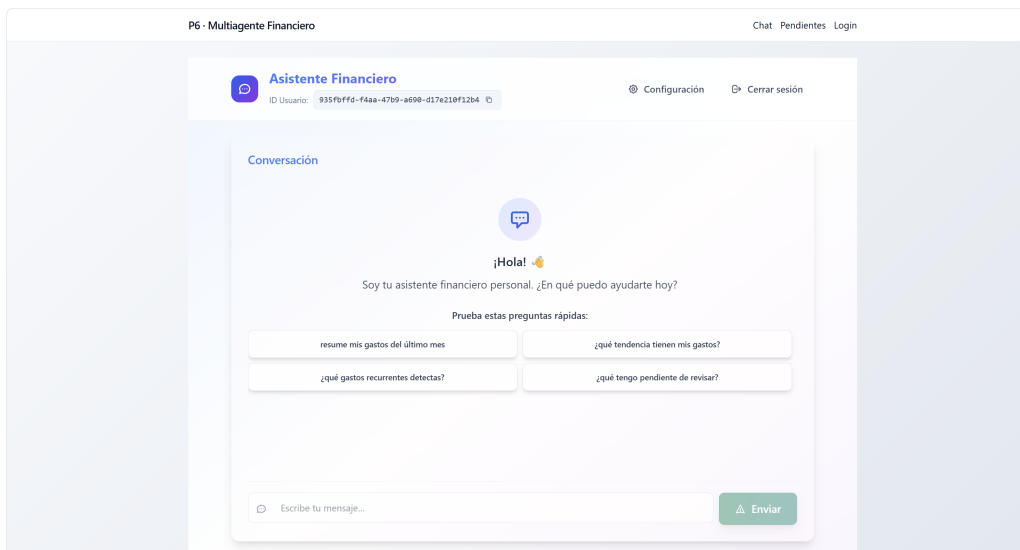


Figura 4: Pantalla de chat: el orquestador narra en español el resultado devuelto por el sub-agente correspondiente.

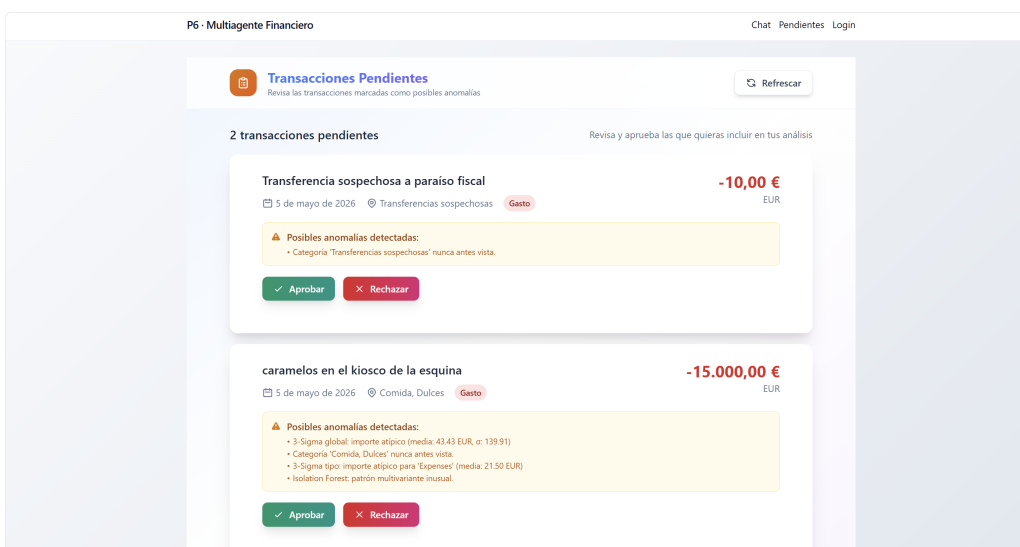


Figura 5: Vista de transacciones pendientes con acciones de aprobar y rechazar.

5. Conclusiones

La Práctica 6 ha cumplido el objetivo de **integrar bajo una arquitectura coherente** las cinco prácticas anteriores, transformando un conjunto disperso de prototipos en una aplicación completa, persistente, multiusuario, accesible vía API y frontal web. La principal conclusión técnica es que **separar razonamiento (LLM) y ejecución (modelos clásicos)** dentro del paradigma multiagente proporciona tres beneficios concretos: latencias predecibles en los sub-agentes, ausencia de alucinaciones numéricas en los *analytics*, y una superficie de *prompting* reducida y manejable en el orquestador. La regla complementaria de que *sólo el orquestador habla con el usuario* simplifica radicalmente el control de errores y elimina errores de coordinación.

Las mayores dificultades se concentraron en cuatro áreas: (i) la coexistencia de tres *frameworks* de *deep learning* (PaddlePaddle, PyTorch, TensorFlow), resuelta eliminando TensorFlow y anclando todo el stack a NumPy 1.26.4; (ii) los bucles infinitos del orquestador, resueltos con la separación `_route/_narrate`; (iii) el *schema drift* entre versiones del esquema relacional, resuelto con migraciones ligeras idempotentes; y (iv) la incompatibilidad entre `passlib` y `bcrypt` 4.x, resuelta migrando a la API nativa. Cada uno de estos problemas se detectó en pruebas.

Las líneas de trabajo futuro identificadas son: integración de Alembic para gestionar migraciones complejas (renames, cambios de tipo); persistencia CRUD completa de objetivos (tabla `goals`); *fine-tuning* del clasificador de *liveness*; Uso y análisis de logs. En conjunto, P6 deja una base sobre la que iterar y un patrón replicable para futuros sistemas multiagente que combinen LLM con modelos clásicos.

Referencias Bibliográficas

- [1] LangChain AI, *LangGraph: Building stateful, multi-actor applications with LLMs*, <https://langchain-ai.github.io/langgraph/>, Accedido en mayo de 2026, 2024.
- [2] PaddlePaddle Authors, *PaddleOCR: Awesome multilingual OCR toolkits based on PaddlePaddle*, <https://github.com/PaddlePaddle/PaddleOCR>, Accedido en mayo de 2026, 2024.
- [3] K. Zhang, Z. Zhang, Z. Li e Y. Qiao, «Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks,» *IEEE Signal Processing Letters*, vol. 23, n.º 10, págs. 1499-1503, 2016.
- [4] F. Schroff, D. Kalenichenko y J. Philbin, «FaceNet: A Unified Embedding for Face Recognition and Clustering,» en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, págs. 815-823.
- [5] T. Guo et al., «Large Language Model based Multi-Agents: A Survey of Progress and Challenges,» *arXiv preprint arXiv:2402.01680*, 2024.
- [6] J. Deng, J. Guo, N. Xue y S. Zafeiriou, «ArcFace: Additive Angular Margin Loss for Deep Face Recognition,» en *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, págs. 4690-4699.